

Aula 2: O que uma rede consegue representar

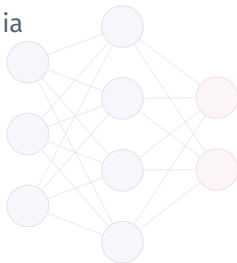
Funções booleanas • Classificadores • Aproximação universal • Profundidade

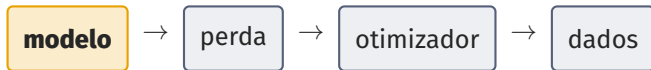
Prof. Marcelo Keese Albertini

GBC073 — Inteligência Computacional • Universidade Federal de Uberlândia

Traduzida e adaptada da Lecture 2 de 11-785 (CMU, Spring 2026), Bhiksha Raj

2026





Antes de treinar (próxima aula): existe algum conjunto de pesos que computa a função desejada?

Roteiro:

- ▶ MLPs como funções **booleanas**
- ▶ MLPs como **classificadores**
- ▶ MLPs como **aproximadores** de funções contínuas
- ▶ Em cada parte: o papel da **profundidade**

Intuição

Uma camada oculta representa tudo, mas pode exigir largura **exponencial**. Profundidade reduz esse custo.

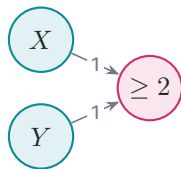
PARTE 1

MLPs como funções booleanas universais

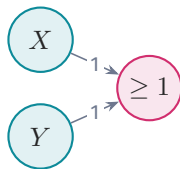


O neurônio de limiar é uma porta lógica

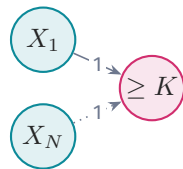
Com entradas em $\{0, 1\}$ e degrau, escolher w e o limiar T dá as portas clássicas — e a de maioria:



AND: limiar 2



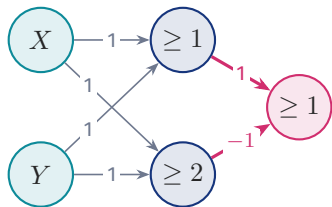
OR: limiar 1



maioria: K de N

Intuição

A porta de maioria custa **um** neurônio; em circuitos AND/OR/NOT de profundidade fixa, custa tamanho exponencial. Neurônios de limiar são portas mais fortes que as booleanas.



XOR = OR e **não** AND: 3 neurônios

Cuidado

Da aula 0: um único perceptron não computa o XOR — as classes não são linearmente separáveis.

- ▶ A camada oculta re-representa a entrada (computa OR e AND)
- ▶ Para a saída, o problema fica linear

Uma camada oculta representa qualquer tabela-verdade

Toda função booleana é uma tabela; toda tabela vira *forma normal disjuntiva*: um OR de termos AND, um termo por linha com saída 1.

$$Y = \underbrace{\bar{X}_1 \bar{X}_2 X_3 X_4 \bar{X}_5}_{\text{um neurônio AND}} + \bar{X}_1 X_2 \bar{X}_3 X_4 X_5 + \cdots + \underbrace{X_1 X_2 \bar{X}_3 \bar{X}_4 X_5}_{\text{um neurônio por linha}}$$

Camada oculta = os ANDs; saída = o OR.

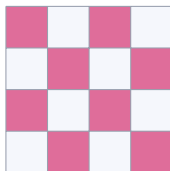
Teorema — MLP como função booleana universal

Uma MLP com uma camada oculta de neurônios de limiar representa exatamente qualquer função booleana de N variáveis.

A pergunta que resta: quantos neurônios?

O pior caso: paridade

O XOR de N variáveis é o mapa de Karnaugh que não se reduz



nenhum agrupamento possível

Metade das 2^N linhas tem saída 1; nenhum AND cobre duas delas.

- ▶ **Uma camada oculta:** 2^{N-1} neurônios
- ▶ **Árvore de XORs:** $3(N-1)$ neurônios, em $2 \log_2 N$ camadas

$$2^{N-1} \longrightarrow 3(N-1)$$

Reduzir a profundidade abaixo do mínimo reverte o ganho: a camada K passa a precisar de $\mathcal{O}(2^{cN})$ neurônios.

Cuidado

Shannon: quase toda função booleana de N variáveis exige mais de $2^N/N$ portas, **independentemente da profundidade**.

- ▶ Profundidade só comprime funções com estrutura *composicional*, como a paridade
- ▶ Hipótese do *deep learning*: visão, fala e linguagem têm essa estrutura
- ▶ O custo real é o número de **conexões** (pesos), não de neurônios — exponencial em neurônios é pior ainda em pesos

Para discutir em aula

Uma camada oculta representa qualquer função booleana. Logo, profundidade não muda o que a rede pode computar.

Verdadeiro / Falso

Para discutir em aula

Uma camada oculta representa qualquer função booleana. Logo, profundidade não muda o que a rede pode computar.

Verdadeiro / Falso

Fechamento

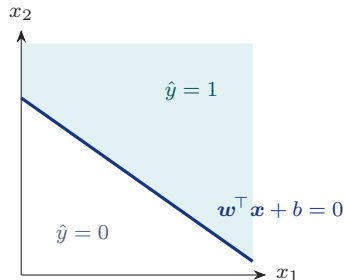
Falso no que importa. O conjunto de funções é o mesmo, mas o custo não: achar pode inflar a largura exponencialmente (paridade: $3(N-1) \rightarrow 2^{N-1}$). Expressividade se mede **sob restrição de tamanho**.

PARTE 2

MLPs como classificadores universais



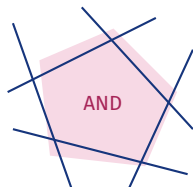
No contínuo, o perceptron é um hiperplano



Definição — Classificador linear

Com entradas reais, o neurônio de limiar dispara em um semiespaço: $\hat{y} = 1[w^\top x + b \geq 0]$. A fronteira é um hiperplano em $\mathbb{R}^n$.

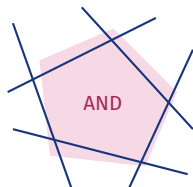
- ▶ As portas booleanas são este desenho com entradas em $\{0, 1\}^2$
- ▶ Pergunta: que regiões de decisão uma rede de hiperplanos desenha?



5 hiperplanos $\rightarrow$ pentágono

A construção clássica tem três camadas:

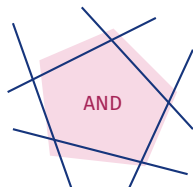
- **Camada 1** — hiperplanos: um neurônio por lado



5 hiperplanos $\rightarrow$ pentágono

A construção clássica tem três camadas:

- ▶ **Camada 1** — hiperplanos: um neurônio por lado
- ▶ **Camada 2** — AND: interseção $\rightarrow$ polítopo convexo



5 hiperplanos $\rightarrow$ pentágono

A construção clássica tem três camadas:

- ▶ **Camada 1** — hiperplanos: um neurônio por lado
- ▶ **Camada 2** — AND: interseção $\rightarrow$ polítopo convexo
- ▶ **Camada 3** — OR: união de polítopos $\rightarrow$ região **arbitrária**, até desconexa e com buracos

Intuição

Qualquer região pode ser ladrilhada por polígonos convexos: duas camadas ocultas desenharam qualquer fronteira de decisão.

O truque do cilindro: uma camada oculta também basta

Some os N lados de um polígono de muitos lados, sem AND:

$$\sum_{i=1}^N y_i \approx \begin{cases} N & \text{dentro do polígono} \\ N/2 & \text{fora, em quase todo lugar} \end{cases} \quad (N \rightarrow \infty : \text{um cilindro})$$

- ▶ Subtraia $N/2$: o cilindro vale $N/2$ dentro, 0 fora
- ▶ Some cilindros centrados onde quiser e limiarize: qualquer região
- ▶ Tudo em uma camada oculta $\Rightarrow$ MLPs rasas são classificadores universais

Cuidado

Cada cilindro exige $N \rightarrow \infty$ neurônios. A construção profunda (retas $\rightarrow$ AND $\rightarrow$ OR) faz o mesmo com neurônios finitos.

Pior caso 2D: retas em grade, regiões alternadas (uma paridade contínua).
Neurônios para representar exatamente:

	rede rasa (2 camadas)	rede profunda (árvore XOR)
grade 4×4 (16 retas)	57 neurônios	46 neurônios
grade 8×8 (64 retas)	609 neurônios	190 neurônios
regra geral (N retas)	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$

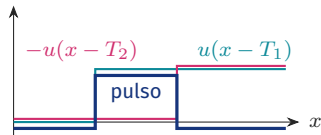
- ▶ Em 2D o custo é quadrático porque há só 2 dimensões independentes
- ▶ Em dimensão D , o pior caso raso cresce como $\mathcal{O}(N^D / (D - 1)!)$: exponencial na dimensão
- ▶ A rede profunda depende só de N , o número de detectores lineares

PARTE 3

MLPs como aproximadores universais



Um pulso com dois degraus; uma função com muitos pulsos



dois neurônios + soma = um pulso

Algoritmo — Aproximação por pulsos (1D)

- ▶ Fatie o domínio em janelas $[T_i, T_{i+1}]$
- ▶ Um pulso por janela, escalado por $h_i \approx f(T_i)$
- ▶ Some: $\hat{f}(x) = \sum_i h_i \text{pulso}_i(x)$

Janelas mais finas $\Rightarrow$ erro arbitrariamente pequeno. Em $\mathbb{R}^n$, troque pulsos por cilindros.

Teorema — Versão que provamos (1D, sigmoide)

Sejam $f: [a, b] \rightarrow \mathbb{R}$ contínua e $\epsilon > 0$. Existem K , coeficientes c_i e limiares T_i tais que

$$g(x) = \sum_i c_i \sigma(k(x - T_i)) \quad \text{satisfaz} \quad \sup_{x \in [a, b]} |f(x) - g(x)| < \epsilon.$$

Três passos, com orçamento de erro $\epsilon/2 + \epsilon/2$:

1. Continuidade uniforme $\rightarrow$ partição de $[a, b]$ em janelas curtas
2. Rede de **degraus** constante por janela: erro $\leq \epsilon/2$
3. Degrau $\rightarrow$ sigmoide com inclinação k grande: erro extra $< \epsilon/2$

Definição — Continuidade uniforme (Heine–Cantor)

f contínua no compacto $[a, b]$ é uniformemente contínua: para todo $\epsilon > 0$ existe $\delta > 0$ tal que $|x - x'| < \delta \Rightarrow |f(x) - f(x')| < \epsilon/2$.

- ▶ O δ vale para o intervalo **inteiro** — é isso que o compacto compra; em $\mathbb{R}$ todo, $f(x) = x^2$ já não teria δ único
- ▶ Particione: $a = T_0 < T_1 < \dots < T_K = b$ com $T_{i+1} - T_i < \delta$, ou seja, $K = \lceil (b - a)/\delta \rceil$ janelas
- ▶ Dentro de cada janela, f varia menos que $\epsilon/2$

$$x \in [T_i, T_{i+1}) \Rightarrow |f(x) - f(T_i)| < \epsilon/2$$

Com o pulso do slide anterior, $\text{pulso}_i(x) = u(x - T_i) - u(x - T_{i+1})$, defina

$$g_u(x) = \sum_{i=0}^{K-1} f(T_i) \text{pulso}_i(x).$$

- ▶ Os pulsos não se sobrepõem: em $[T_i, T_{i+1})$ vale $g_u(x) = f(T_i)$, constante
- ▶ Pelo Passo 1: $|f(x) - g_u(x)| = |f(x) - f(T_i)| < \epsilon/2$ em toda janela
- ▶ Arquitetura: **uma** camada oculta com $2K$ degraus e saída linear (K degraus na forma telescópica $g_u = f(T_0) + \sum_i [f(T_i) - f(T_{i-1})] u(x - T_i)$)

$$\sup_{x \in [a,b]} |f(x) - g_u(x)| \leq \epsilon/2$$

Se a ativação permitida fosse o degrau, a prova acabaria aqui.

Passo 3: trocar o degrau pela sigmoide

Substitua cada $u(x - T_i)$ por $\sigma(k(x - T_i))$, com o mesmo c_i ; chame o resultado de g . Note que $\sigma(kz) \rightarrow u(z)$ quando $k \rightarrow \infty$, para todo $z \neq 0$.

- ▶ **Longe dos limiares** ($|x - T_i| \geq \Delta$ para todo i): cada termo erra no máximo $\sigma(-k\Delta)$; o erro total é $\leq K \max_i |c_i| \sigma(-k\Delta)$, que $\rightarrow 0$ em k . Escolha k que o torne $< \epsilon/2$
- ▶ **Perto de um limiar** T_i : g fica, a menos desse mesmo erro, entre os patamares vizinhos $f(T_{i-1})$ e $f(T_i)$ — e ambos distam menos de $\epsilon/2$ de $f(x)$, pois a janela é curta (Passo 1)

Nos dois casos, $|f(x) - g(x)| < \epsilon/2 + \epsilon/2 = \epsilon$.

$$\sup_{x \in [a,b]} |f(x) - g(x)| < \epsilon \quad \blacksquare$$

- ▶ $\mathbb{R}^n$, **construtivo**: troque pulsos por torres — produto de janelas por coordenada. Com uma camada extra (soma dos $2n$ degraus + limiar em $2n - \frac{1}{2}$), a torre fica exata; o resto do argumento é idêntico
- ▶ $\mathbb{R}^n$, **uma camada oculta (Cybenko)**: prova por contradição, não construtiva. Se o fecho de $\text{span}\{\sigma(\mathbf{w}^\top \mathbf{x} + b)\}$ não fosse todo o $C(K)$, Hahn–Banach daria uma medida $\mu \neq 0$ com $\int \sigma(\mathbf{w}^\top \mathbf{x} + b) d\mu = 0$ para todos $\mathbf{w}, b$; como a sigmoide é *discriminatória*, isso força $\mu = 0$ — contradição
- ▶ **Leshno et al.**: a ativação só precisa ser **não polinomial** — um polinômio de grau d gera apenas polinômios de grau d , e esses não são densos

A versão construtiva diz o preço ($K \sim (b - a)/\delta$) por dimensão — exponencial em n ; a de Cybenko diz só que existe.

Teorema — Aproximação universal (Cybenko 1989; Hornik 1989; Leshno 1993)

Uma rede com uma camada oculta e ativação não polinomial aproxima qualquer função contínua em um compacto de $\mathbb{R}^n$ com precisão arbitrária.

O QUE ELE GARANTE

- ▶ Existe rede com erro $< \epsilon$, para qualquer ϵ
- ▶ Vale para sigmoide, tanh, ReLU, ...
- ▶ A arquitetura MLP não é o gargalo teórico

O QUE ELE NÃO DIZ

- ▶ Quantos neurônios (pode ser astronômico)
- ▶ Como achar os pesos
- ▶ Nada sobre dados finitos ou generalização

Existência $\neq$ aprendizado: otimização e generalização são as próximas semanas.

Com $\text{ReLU}(z) = \max(0, z)$, a rede computa uma função **linear por partes**: o domínio se parte em regiões, afim em cada uma.

- ▶ Expressividade $\approx$ número de regiões
- ▶ Largura n , profundidade L , entrada n_0 (Montúfar *et al.*, 2014):

$$\# \text{regiões} \geq \Omega \left(\left(\frac{n}{n_0} \right)^{(L-1)n_0} n^{n_0} \right)$$

Intuição

Exponencial em L , polinomial em n : cada camada reusa e recombina as regiões da anterior; largura só acrescenta regiões uma a uma.

É o resultado da paridade, agora no contínuo.

No PyTorch: largura vs. profundidade

```
import torch, torch.nn as nn

def mlp(larguras):
    cam = []
    for a, b in zip(larguras[:-1], larguras[1:]):
        cam += [nn.Linear(a, b), nn.ReLU()]
    return nn.Sequential(*cam[:-1])    # sem ReLU na saída

raso      = mlp([1, 420, 1])           # 1 camada oculta: 1261 parametros
profundo  = mlp([1, 24, 24, 24, 24, 1]) # 4 camadas ocultas: 1873 parametros

x = torch.linspace(-3, 3, 512).unsqueeze(1)
y = torch.sign(torch.sin(4 * x))       # onda quadrada: muitos "degraus"

for modelo in (raso, profundo):
    opt = torch.optim.Adam(modelo.parameters(), lr=1e-3)
    for passo in range(3000):
        perda = ((modelo(x) - y) ** 2).mean()
        opt.zero_grad(); perda.backward(); opt.step()
```

Mesmo orçamento de parâmetros. Plote as duas saídas: a profunda ajusta os degraus; a rasa arredonda os cantos.

No PyTorch: paridade, raso vs. profundo

```
N = 12 # paridade de 12 bits
X = torch.randint(0, 2, (8192, N)).float()
y = (X.sum(1) % 2).unsqueeze(1) # XOR de todos os bits

raso = mlp([N, 512, 1])
profundo = mlp([N, 32, 32, 32, 32, 1])

perda_fn = nn.BCEWithLogitsLoss()
for modelo in (raso, profundo):
    opt = torch.optim.Adam(modelo.parameters(), lr=1e-3)
    for passo in range(5000):
        perda = perda_fn(modelo(X), y)
        opt.zero_grad(); perda.backward(); opt.step()
    acerto = ((modelo(X) > 0) == y.bool()).float().mean()
```

Meça também em dados de teste. A profunda pode compor XORs — a árvore de $3(N - 1)$ neurônios, agora encontrada pelo gradiente. Nem sempre encontra: representável $\neq$ treinável.

Uma rede representa a função-alvo só se tiver capacidade: largura e profundidade suficientes. No tabuleiro de 16 retas:

- ▶ Menos de 16 neurônios na 1ª camada $\Rightarrow$ impossível representar o padrão: faltam retas
- ▶ Com degrau, a 1ª camada **apaga informação**: regiões distintas colapsam na mesma saída binária, e camadas posteriores não recuperam
- ▶ Ativações graduadas (sigmoide, ReLU) passam informação adiante: camadas seguintes corrigem o que a primeira perdeu

Intuição

Segundo argumento contra o degrau, independente do gradiente (aula 0): ele também limita a *representação*.

A unidade de base radial responde à **distância** de um centro, não a um produto interno:

$$z = e^{-\beta \|x - \mu\|^2}$$

- ▶ Um cilindro por unidade — o que custava $N \rightarrow \infty$ perceptrons
- ▶ Parente direto dos métodos de *kernel*

Por que ficou para trás

- ▶ Local demais: não compõe representações em camadas
- ▶ Escala mal com a dimensão da entrada
- ▶ A resposta por similaridade reaparece na *atenção* (fim do semestre)

Para discutir em aula

Pelo teorema da aproximação universal, uma rede rasa larga o bastante, treinada com dados suficientes, aprende qualquer função.

Verdadeiro / Falso

Para discutir em aula

Pelo teorema da aproximação universal, uma rede rasa larga o bastante, treinada com dados suficientes, aprende qualquer função.

Verdadeiro / Falso

Fechamento

Falso. O teorema garante que a rede existe; não garante que o gradiente a encontra, nem que dados finitos a selecionam. Representar, otimizar e generalizar são três problemas distintos — esta aula tratou só do primeiro.

- ▶ MLPs são universais: funções booleanas, classificadores e aproximadores — mesmo com uma camada oculta
- ▶ A versão rasa pode custar largura exponencial (paridade, dimensão da entrada)
- ▶ Profundidade: $\mathcal{O}(2^N) \rightarrow \mathcal{O}(N)$ na paridade; regiões lineares exponenciais em L na ReLU
- ▶ Capacidade insuficiente $\Rightarrow$ não representa; ativações graduadas preservam informação entre camadas

Próxima aula

Treinamento: aprendizado como minimização de risco empírico e descida de gradiente.

Fonte: Lecture 2, 11-785 (CMU S26).
Leitura: Nielsen, cap. 4.

Exercício 1 — Construção

(a) Desenhe a MLP de limiar (pesos e limiares) que computa a paridade de 4 bits como árvore de XORs; conte neurônios e pesos. (b) Escreva a DNF da mesma função e conte os neurônios da versão de uma camada oculta. (c) Repita as contagens para $N = 20$ bits.

Exercício 2 — Código

Rode o experimento da paridade medindo acurácia em teste (bits novos), com $N \in \{6, 10, 14\}$ e larguras variadas da rede rasa. Em que ponto a rasa deixa de generalizar? A profunda sempre converge? Relacione com “representável $\neq$ treinável”.

Leitura: Nielsen, cap. 4 • Montúfar *et al.* (2014), seções 1–2 • Telgarsky (2016), separação profundidade/largura.